

NAAN MUDHALVAN PROJECT

LegalEase: AI-Powered Legal Document Generator

Testing and Verifying Local Deployment

PROJECT REPORT

Project Domain: Artificial Intelligence / Natural Language Processing / Web Application

Project Type: Academic Mini Project / Capstone Project

Submitted as part of the Naan Mudhalvan Project Work

Student Name: _____

Register Number: _____

Department / Class: _____

Institution: _____

Academic Year: 2026–2027

CERTIFICATE

This is to certify that the project titled “**LegalEase: AI-Powered Legal Document Generator**”, with the subtitle “**Testing and Verifying Local Deployment**”, has been prepared as part of the Naan Mudhalvan project work. The project presents the design, development, testing, and local deployment verification of an AI-assisted application for generating structured legal-document drafts from user-provided information. The work has been prepared for academic and demonstration purposes.

DECLARATION

I hereby declare that this project report titled “**LegalEase: AI-Powered Legal Document Generator**” is prepared for academic project submission. The report describes the proposed system, implementation approach, testing strategy, local deployment procedure, observations, limitations, and future enhancements. The generated documents are intended as drafts and educational outputs and should not be treated as a substitute for professional legal advice or legal review.

ACKNOWLEDGEMENT

I sincerely thank my institution, faculty members, project guide, and the Naan Mudhalvan programme for providing the opportunity to work on an application-oriented technology project. I also thank everyone who provided suggestions during requirement analysis, interface design, testing, and documentation. The project helped me understand how artificial intelligence, web application development, document generation, and software testing can be combined to solve a practical problem.

ABSTRACT

LegalEase is an AI-powered legal document generator designed to help users create structured first-draft documents from guided inputs. The system combines a simple user interface, document templates, validation logic, and an AI-assisted generation layer. The project focuses particularly on **testing and verifying local deployment**, ensuring that the application can be installed, configured, executed, tested, and demonstrated on a local development environment. The system is designed with safety in mind: it uses clear input fields, validation, disclaimers, and review-oriented output. The project demonstrates a practical workflow from requirements to implementation, test cases, deployment verification, and future improvement.

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. Existing System
6. Proposed System
7. Feasibility Study
8. Requirements Analysis
9. System Architecture
10. Module Description
11. User Interface Design
12. AI and NLP Approach
13. Document Generation Workflow
14. Data Validation and Safety
15. Database and Data Management
16. Local Deployment Environment
17. Installation Procedure
18. Configuration
19. Testing Strategy
20. Test Cases
21. Unit Testing
22. Integration Testing
23. System Testing
24. Usability Testing
25. Security Testing
26. Local Deployment Verification
27. Results and Observations
28. Limitations
29. Future Enhancements
30. Conclusion
31. References
32. Appendix

1. INTRODUCTION

1.1 Background

Artificial Intelligence has become an important technology for automating repetitive information-processing tasks. In document-oriented workflows, AI can assist users by converting structured information into readable drafts. Legal documents are often formal, repetitive, and sensitive to wording, which makes them suitable for template-based automation combined with controlled AI assistance. LegalEase explores this idea through an academic web application.

1.2 Project Overview

LegalEase provides a guided workflow in which the user selects a document category, enters required information, validates the input, and generates a structured draft. The application is designed to separate user input, processing, generation, and output. This separation makes the project easier to test and deploy locally.

1.3 Need for the Project

Many users find formal document drafting difficult because they may not know the expected structure, terminology, or order of information. A guided generator can reduce formatting effort and provide a consistent starting point. The system is not intended to replace lawyers; instead, it demonstrates how technology can support document preparation while keeping human review in the process.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

2. PROBLEM STATEMENT

2.1 Problem

Preparing a formal document manually can involve repeated typing, inconsistent formatting, missing fields, and difficulty maintaining a standard structure. For an academic project, there is also a need to demonstrate how an AI-enabled application can be reliably installed, executed, and tested in a local environment.

2.2 Proposed Problem Solution

LegalEase addresses the problem by collecting information through structured fields and generating a consistent draft. Validation rules identify missing or invalid values before generation. Local deployment verification ensures that the same application can be started and tested on the developer machine with documented configuration steps.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

3. OBJECTIVES

3.1 Primary Objectives

The main objective is to develop an AI-assisted document generation prototype with a clear user workflow. Another major objective is to test the application and verify that the local deployment works correctly.

3.2 Specific Objectives

The project aims to provide a simple interface; accept structured inputs; validate mandatory fields; generate standardized drafts; integrate an AI layer where appropriate; handle errors gracefully; test individual modules; perform integration and system testing; verify local deployment; and document the results.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

4. SCOPE OF THE PROJECT

4.1 Functional Scope

The prototype can support selected document categories such as notices, agreements, declarations, applications, and other academic demonstration templates. The exact categories can be extended without changing the overall architecture.

4.2 Technical Scope

The project covers frontend interaction, backend processing, prompt or template orchestration, document generation, validation, logging, testing, and local deployment. It may be implemented using common web technologies such as HTML, CSS, JavaScript, Python or Node.js, depending on the chosen development stack.

4.3 User Scope

The intended academic users are students, demonstrators, and prototype evaluators. Any real-world legal use requires review by an appropriately qualified professional and consideration of the laws applicable to the user's jurisdiction.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

5. EXISTING SYSTEM

5.1 Manual Drafting

Traditional drafting commonly depends on manually entering information into a document editor. This can be slow for repetitive documents and may result in inconsistent formatting.

5.2 Template-Based Drafting

Predefined templates improve consistency but can become difficult to maintain when many document types or conditional clauses are required.

5.3 Limitations

A purely manual approach may have more opportunities for missing fields, inconsistent wording, and formatting errors. LegalEase attempts to reduce these issues through guided input and automated checks, while still keeping human review necessary.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

6. PROPOSED SYSTEM

6.1 System Description

The proposed system combines a user interface, validation module, document-generation engine, optional AI service, and output module. The system follows a controlled flow rather than allowing unrestricted generation.

6.2 Key Features

Key features include document-type selection, structured data entry, validation, AI-assisted drafting, standardized formatting, preview or output generation, error messages, and local deployment support.

6.3 Advantages

The prototype can improve consistency, reduce repetitive typing, make the generation process easier to demonstrate, and provide a clear testing boundary between input, processing, and output.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

7. FEASIBILITY STUDY

7.1 Technical Feasibility

The application can be implemented using widely available programming languages, web frameworks, document libraries, and development tools. Local execution is feasible on a standard student development computer.

7.2 Operational Feasibility

The guided interface is intended to be simple enough for demonstration. Users provide information through fields instead of writing an entire document from scratch.

7.3 Economic Feasibility

An academic prototype can be developed using open-source development tools. If an external AI API is used, usage limits and service costs must be considered during deployment.

7.4 Schedule Feasibility

The project can be divided into requirement analysis, UI development, backend development, AI integration, testing, deployment verification, and documentation phases.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

8. REQUIREMENTS ANALYSIS

8.1 Functional Requirements

The system shall allow document selection, collect required fields, validate inputs, initiate generation, display or save the generated draft, and report errors. The system should also prevent generation when critical inputs are missing.

8.2 Non-Functional Requirements

The application should be usable, maintainable, reasonably responsive, secure for a prototype, and easy to run locally. Configuration should be separated from application code where possible.

8.3 Hardware Requirements

A typical development computer with a modern processor, sufficient RAM, storage, keyboard, display, and network access when an external AI service is used is adequate for the prototype.

8.4 Software Requirements

The exact stack may include a code editor, Git, a runtime such as Python or Node.js, a web browser, package manager, document-generation library, and optional AI API credentials.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

9. SYSTEM ARCHITECTURE

9.1 Architecture Overview

The system follows a layered architecture. The presentation layer collects inputs; the application layer validates and processes them; the generation layer creates content; and the output layer renders the final draft. Configuration and logging support the application across development and testing.

9.2 Data Flow

User → Document Type Selection → Input Form → Validation → Generation Engine → AI/Template Processing → Draft Assembly → Output/Preview → Human Review.

9.3 Error Flow

Invalid input or service failure is captured by the error-handling layer. The user receives a readable message instead of an incomplete document. Logs can be used by developers to diagnose the underlying issue.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

10. MODULE DESCRIPTION

10.1 User Module

Provides the interface for selecting document type and entering information. It should show required fields clearly and guide the user through the process.

10.2 Validation Module

Checks required values, data formats, length constraints, and incompatible combinations before generation.

10.3 AI Generation Module

Transforms validated information into a structured draft using a controlled prompt or generation strategy. It should be configured to avoid presenting the result as professional legal advice.

10.4 Template Module

Stores document structures, headings, placeholders, and formatting rules. Templates provide a stable fallback and help keep outputs consistent.

10.5 Output Module

Displays the generated draft and can provide a copy, download, or document export depending on the implementation.

10.6 Logging Module

Records application events such as startup, validation failures, generation status, and errors without unnecessarily storing sensitive user content.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

11. USER INTERFACE DESIGN

11.1 Design Principles

The interface should be clean, readable, and task-oriented. Each screen should communicate what information is required and what happens next.

11.2 Input Form

The form can include document type, party information, dates, purpose, terms, and other fields appropriate to the selected document. Required fields should be visually identifiable.

11.3 Preview Screen

A preview stage allows users to inspect the generated draft before saving or using it. A clear disclaimer should remind users that the output requires human review.

11.4 Error Messages

Messages should explain what needs to be corrected rather than exposing technical stack traces. For example, the interface can state that a required field is missing or that generation could not be completed.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

12. AI AND NLP APPROACH

12.1 Role of AI

AI is used as an assistance layer for converting structured information into natural-language draft content. The system should constrain the AI with document type, required sections, and formatting expectations.

12.2 Prompt Structure

A controlled prompt can specify the document type, user-provided facts, required sections, prohibited assumptions, output format, and a reminder that the result is a draft requiring review.

12.3 Hallucination Control

The application should avoid asking the model to invent facts. User inputs should be clearly separated from instructions, and generated content should be checked against the supplied information.

12.4 Human Review

Human review is an essential part of the workflow. Users should verify names, dates, amounts, obligations, jurisdiction, references, and other important information before relying on a document.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

13. DOCUMENT GENERATION WORKFLOW

13.1 Step 1 – Select Document

The user selects the type of document from the available options.

13.2 Step 2 – Enter Information

The user completes structured fields. The application should request only information relevant to the selected document type.

13.3 Step 3 – Validate

The validation module checks completeness and basic consistency.

13.4 Step 4 – Generate

Validated data is sent to the template engine and, where configured, the AI generation service.

13.5 Step 5 – Assemble

Generated sections are combined with headings, formatting, and required notices.

13.6 Step 6 – Review

The user previews the result and checks it for accuracy before export or further editing.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

14. DATA VALIDATION AND SAFETY

14.1 Input Validation

Validation reduces accidental errors by checking empty fields, invalid dates, incorrect email formats where applicable, excessive input length, and other simple constraints.

14.2 Privacy

A prototype should avoid collecting unnecessary personal information. Sensitive values should not be written to logs unless required for controlled testing.

14.3 Safety Notice

LegalEase is an academic prototype. Generated content should be treated as a draft and reviewed by a qualified legal professional when legal consequences are involved.

14.4 Output Verification

Important factual fields should be compared with the original input. Automated checks can identify differences, but they should not be treated as a complete legal review.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

15. DATABASE AND DATA MANAGEMENT

15.1 Data Model

A simple model can include document type, input metadata, generation status, timestamps, and output location. For a prototype, temporary in-memory storage or a lightweight database can be sufficient.

15.2 Storage Strategy

The system should store only the information needed for the intended workflow. If persistent storage is enabled, access control and secure configuration should be considered.

15.3 Backup and Recovery

Project source code and configuration templates should be backed up through a version-control system. Secrets should not be committed to source control.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

16. LOCAL DEPLOYMENT ENVIRONMENT

16.1 Purpose

Local deployment provides a controlled environment for testing the application before any hosted deployment. It allows developers to verify dependencies, configuration, routes, generation logic, and error handling.

16.2 Environment Components

Typical components include the operating system, runtime, package manager, source code, dependency files, environment variables, local server, browser, and optional AI credentials.

16.3 Deployment Principle

The deployment process should be reproducible: install dependencies, configure environment variables, start the server, access the local URL, and execute the documented test cases.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

17. INSTALLATION PROCEDURE

17.1 Source Setup

Obtain the project source code and place it in a working directory. Open the project in the selected development environment.

17.2 Dependency Installation

Install the packages listed by the project dependency file. Verify that installation completes without unresolved dependency errors.

17.3 Environment Configuration

Create the required environment configuration from a safe template. Add API credentials only through local environment variables or a secure secret mechanism.

17.4 Start Application

Run the project using its configured development command. Confirm that the local server starts and reports the expected host and port.

17.5 Browser Verification

Open the local application address in a browser and verify that the home page loads without console or server errors.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

18. CONFIGURATION

18.1 Configuration Variables

Typical configuration values include application mode, local host/port, AI provider settings, model name, output directory, and logging level. Actual names depend on implementation.

18.2 Secret Management

API keys and passwords should never be hard-coded in source files. A local environment file may be used during development, while the file itself should remain excluded from version control.

18.3 Configuration Testing

After configuration, test both valid and invalid settings. The application should fail safely and provide a useful diagnostic when a required configuration value is missing.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

19. TESTING STRATEGY

19.1 Testing Goals

Testing verifies that each module behaves correctly and that the complete application works as expected. The strategy covers functionality, validation, integration, usability, security, and local deployment.

19.2 Test Levels

Unit testing checks individual functions. Integration testing checks communication between modules. System testing checks the complete workflow. User-oriented testing checks whether the interface is understandable and usable.

19.3 Test Environment

Tests should be executed in the same local environment used for deployment verification. Browser, runtime version, dependency version, and configuration should be recorded for reproducibility.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

20. TEST CASES

20.1 Test Case Format

Each test case includes an ID, objective, input or precondition, expected result, actual result, and status. The following sample cases can be adapted to the actual implementation.

20.2 Sample Cases

TC01: Application startup – expected: server starts successfully.
TC02: Home page – expected: page loads correctly.
TC03: Empty mandatory field – expected: validation message.
TC04: Valid document input – expected: generation begins.
TC05: AI service unavailable – expected: controlled error or fallback.
TC06: Output formatting – expected: headings and fields appear correctly.
TC07: Refresh/restart – expected: application remains stable.
TC08: Invalid configuration – expected: clear diagnostic without exposing secrets.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

21. UNIT TESTING

21.1 Purpose

Unit testing isolates functions such as validation, template selection, input normalization, prompt construction, and formatting.

21.2 Example Validation Tests

A required-name validator should reject an empty value and accept a valid value. A date validator should reject malformed dates and accept valid dates according to the application's rules.

21.3 Expected Outcome

Unit tests should be repeatable and independent. Failures should identify the function that requires correction rather than requiring the entire application to be debugged.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

22. INTEGRATION TESTING

22.1 Purpose

Integration testing verifies that modules communicate correctly. For LegalEase, this includes the connection between form submission, validation, generation, and output rendering.

22.2 AI Integration

When an AI service is configured, integration testing should verify request construction, response handling, timeout behavior, and error handling. Tests should not expose real secrets.

22.3 Output Integration

The generated content should pass through the formatting and output layer without losing required fields or headings.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

23. SYSTEM TESTING

23.1 End-to-End Test

A complete test begins at document selection and ends with a reviewed draft. The objective is to verify the entire user journey.

23.2 Positive Scenario

Provide valid inputs for a supported document and confirm that the generated draft contains the expected sections and user-provided facts.

23.3 Negative Scenario

Submit incomplete or invalid information and confirm that the application stops generation or provides a safe, understandable response.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

24. USABILITY TESTING

24.1 Usability Goals

The interface should allow a first-time user to understand what to enter, what the system is doing, and what to check before using the result.

24.2 Evaluation Points

Check readability, field labels, navigation, error messages, loading indicators, preview clarity, and accessibility basics such as keyboard navigation and sufficient text contrast.

24.3 Improvement Cycle

Collect feedback, identify confusing steps, update labels or layout, and repeat the test. Usability is treated as an iterative part of development.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

25. SECURITY TESTING

25.1 Input Security

Inputs should be treated as untrusted. The application should validate data and avoid unsafe direct insertion into HTML, database queries, or system commands.

25.2 Secret Security

Credentials should remain outside source code and should not appear in logs, screenshots, or test reports.

25.3 AI Safety

User-provided text should be separated from system instructions. The application should reduce the risk of prompt manipulation and should not request the model to invent legal facts.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

26. LOCAL DEPLOYMENT VERIFICATION

26.1 Verification Checklist

1. Runtime installed. 2. Dependencies installed. 3. Environment configured. 4. Server starts. 5. Local URL responds. 6. Main page loads. 7. Form accepts valid input. 8. Invalid input is rejected. 9. Generation works or fails gracefully. 10. Output can be reviewed. 11. Restart succeeds. 12. No critical console/server errors remain.

26.2 Verification Result

A deployment is considered verified for the academic prototype when the application starts successfully, the major workflow can be completed locally, expected validation behavior is observed, and known failures are handled without crashing the application.

26.3 Evidence

For the final submission, screenshots may be inserted here showing the terminal startup message, local browser page, input form, generated preview, and selected test results. These screenshots should be captured from the student's actual local deployment.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

27. RESULTS AND OBSERVATIONS

27.1 Functional Result

The project demonstrates a complete concept for AI-assisted document generation: structured input, validation, generation, formatting, and review.

27.2 Deployment Result

The local deployment section provides a reproducible procedure for starting the application and verifying its main behavior. The exact runtime output depends on the student's implementation and environment.

27.3 Observations

Controlled templates improve consistency, validation reduces common input mistakes, and explicit human review remains important because AI-generated text may contain errors or omissions.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

28. LIMITATIONS

28.1 Technical Limitations

AI output can vary, external services can be unavailable, and different runtime or package versions can produce environment-specific issues.

28.2 Domain Limitations

The application does not determine whether a document is legally sufficient for a particular jurisdiction or situation. Legal rules can differ by location and change over time.

28.3 Academic Prototype Limitation

The project is primarily a demonstration of technology integration and deployment verification. Production use would require additional security, legal review, compliance controls, monitoring, and quality assurance.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

29. FUTURE ENHANCEMENTS

29.1 Feature Enhancements

Future versions could add more document templates, multilingual support, richer preview, version history, document comparison, collaborative review, and configurable organization templates.

29.2 AI Enhancements

Future work could include retrieval from approved legal sources, stronger factual consistency checks, structured output validation, citation support, and model evaluation using a carefully designed test dataset.

29.3 Deployment Enhancements

The application could be containerized, deployed to a controlled cloud environment, connected to monitoring, and supported by automated CI/CD testing.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

30. CONCLUSION

30.1 Summary

LegalEase demonstrates how AI-assisted generation can be integrated into a structured web application for producing first-draft legal documents. The project covers the complete academic development lifecycle from problem definition and requirements to architecture, implementation concepts, testing, and local deployment verification.

30.2 Final Note

The central outcome of the project is not simply document generation but a repeatable, testable workflow. By combining structured inputs, validation, controlled generation, human review, and deployment checks, the project provides a practical demonstration of responsible AI application development.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

31. REFERENCES

31.1 General References

1. Official documentation of the programming language and runtime used in the implementation.
2. Official documentation of the selected web framework.
3. Official documentation of the selected AI model/API, if an external AI service is used.
4. Documentation for the selected document-generation library.
5. Software testing and secure-development guidance relevant to the implementation.

31.2 Project Documentation

The final report should be supplemented with the actual source-code repository, dependency file, configuration template, test evidence, screenshots, and implementation-specific documentation prepared by the project team.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

32. APPENDIX

32.1 Suggested Screenshot Pages

Insert actual screenshots of: project folder structure; dependency installation; environment configuration template; local server startup; home page; document selection; input form; validation error; generation progress; generated preview; and test result summary.

32.2 Suggested Viva Questions

1. What is LegalEase?
2. Why is AI used in the project?
3. What is local deployment?
4. How do you validate user input?
5. What happens when the AI service is unavailable?
6. What are the limitations of AI-generated legal drafts?
7. What is the difference between unit and integration testing?
8. How are API keys protected?
9. How did you verify local deployment?
10. What future improvements can be added?

32.3 Sample Project Flow

Requirement Analysis → UI Design → Backend Development → Validation → AI/Template Integration → Output Formatting → Testing → Local Deployment → Verification → Documentation.

Project Note: This section is written as a report-ready academic description. Replace generic implementation statements with your actual framework, model, database, screenshots, and measured test results before final submission.

TEST CASE MATRIX

ID	Test	Expected Result	Status
TC01	Start local server	Server starts without critical error	Pass/Fill
TC02	Open home page	UI loads correctly	Pass/Fill
TC03	Empty required field	Validation message shown	Pass/Fill
TC04	Valid input	Generation starts	Pass/Fill
TC05	Invalid input	Request rejected safely	Pass/Fill
TC06	AI/service failure	Graceful error/fallback	Pass/Fill
TC07	Output review	Required sections present	Pass/Fill
TC08	Restart application	Application starts again	Pass/Fill
TC09	Secret check	No credentials exposed	Pass/Fill
TC10	Browser console	No critical errors	Pass/Fill

Fill the Status column using the actual result from your local deployment. Add a screenshot or terminal output as evidence for important cases.

PROJECT IMPLEMENTATION DETAILS

Frontend Technology: _____

Backend Technology: _____

AI Model / Provider: _____

Database / Storage: _____

Document Generation Library: _____

Operating System Used: _____

Runtime Version: _____

Browser Used: _____

Notes / Evidence:

LOCAL DEPLOYMENT RECORD

Project Directory: _____

Install Command: _____

Run Command: _____

Local URL: _____

Deployment Date: _____

Configuration Verified By: _____

Observed Result: _____

Notes / Evidence:

PROJECT GUIDE REVIEW

Guide Name: _____

Department: _____

Review Date: _____

Comments: _____

Signature: _____

Notes / Evidence:
